ASSIGNMENT 2:

SYSTEMIC GAMES WITH EMERGENT SYSTEMS

Designer: Ciaran McIlvaney

TABLE OF CONTENTS

Q.1. JUSTIFY OBJECTS, PROPERTIES, BEHAVIOURS, and RELATIONSHIPS 2
1.0. ASSUMPTIONS: 2
1.1. DIRECTION TUNNEL 2
1.2. TOGGLE SWITCH 3
1.3. ARROW LOCK 4
Q.2. JUSTIFY EMERGENCE, GAMEPLAY POSSIBILITY SPACE 4
2.1. EMERGENT SYSTEM RELATIONSHIPS 4
2.2. SCENERIO JUSTIFICATION 5
2.3. ELEGANT DESIGN AND POSSIBLITY SPACE 6
3.0 SYSTEM IMPLEMENTATION REFLECTION 6

Q.1. JUSTIFY OBJECTS, PROPERTIES, BEHAVIOURS, and RELATIONSHIPS

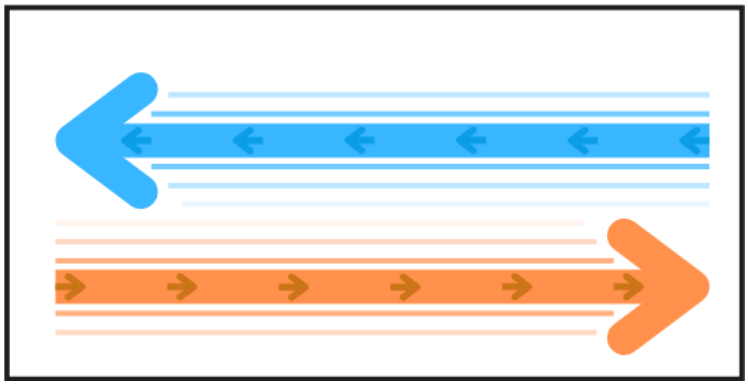
For each mechanic, justify why you designed these specific objects, properties, behaviours, and relationships.

1.0. ASSUMPTIONS:

This design does not document all the foundational gameplay mechanics. It is assumed that standard mechanics commonly found in top-down dungeon crawler games are already present. These include things such as:

- Player movement in four directions
- A close-range melee attack
- A long-range projectile attack
- Basic collisions with walls and hazards

1.1. DIRECTION TUNNEL:



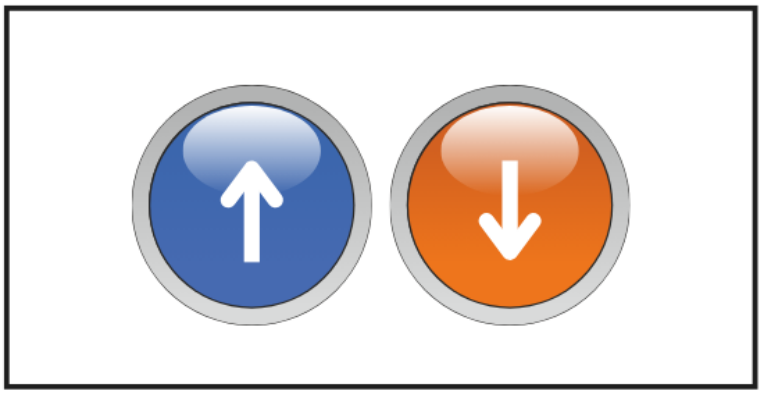
The directional tunnel is the primary systemic object in the design. It is here to control the movement of arrows by applying a directional force. Its key properties is direction, state its in (forward/reversed), and its current status, with these allowing it to serve multiple roles within the system.

The tunnels behaviour of carrying arrows and destroying them when they are misguided shows the consequence of risk, helping reinforce the players understanding of the system. By allowing tunnels to reverse directions through a button input, it becomes clearer to the player what they must do, allowing for multiple puzzle configurations using the same object. This allows a single tunnel

object to serve either as both a constraint and an opportunity depending on the system state.

The tunnel gives strong feedback through visual flow indicators, colour changes, and audio cues allowing players to quickly build a mental model of how the arrow routing works.

1.2. TOGGLE SWITCH:



The toggle switch exists as a systemic controller rather than a fully direct solution. Its purpose is not to unlock the objective itself, but to change the state of other objects in the system. This design encourages secondary problem solving and reinforces systemic thinking.

The switch's binary state (0/1, true/false) makes sure that player actions have consequences, which supports planning and experimentation. By showing a toggle event to all connected Directional Tunnels, the switch creates a many-to-many relationships between objects, adding to the puzzle's difficulty. This also prevents the switch from feeling like a traditional basic lock and key interaction.

Clear visual and audio feedback makes sure that players understand when a state change has happened and how it affects the environment.

1.3. ARROW LOCK:

The Arrow Lock acts as a reward object rewarding the correct system interaction from the player. It is only activated by shooting an arrow into it, preventing brute force solutions and reinforcing the importance of routing and timing.

The Arrow Lock's one time persistence makes sure that success feels meaningful while also showing puzzle completion. Its relationship with the traversal elements such as the rising bridge allows the lock to change the rooms traversal state rather than simply just opening a door.

By taking the arrow upon the activation, the Arrow Lock also reinforces consequence and finality, preventing uncertainty about whether the system state has actually been changed or not.

Q.2. JUSTIFY EMERGENCE, GAMEPLAY POSSIBILITY SPACE

Overall, justify how you used a minimal number of objects with a sufficient number of properties to generate a widely possibility space.

2.1. EMERGENT SYSTEM RELATIONSHIPS:

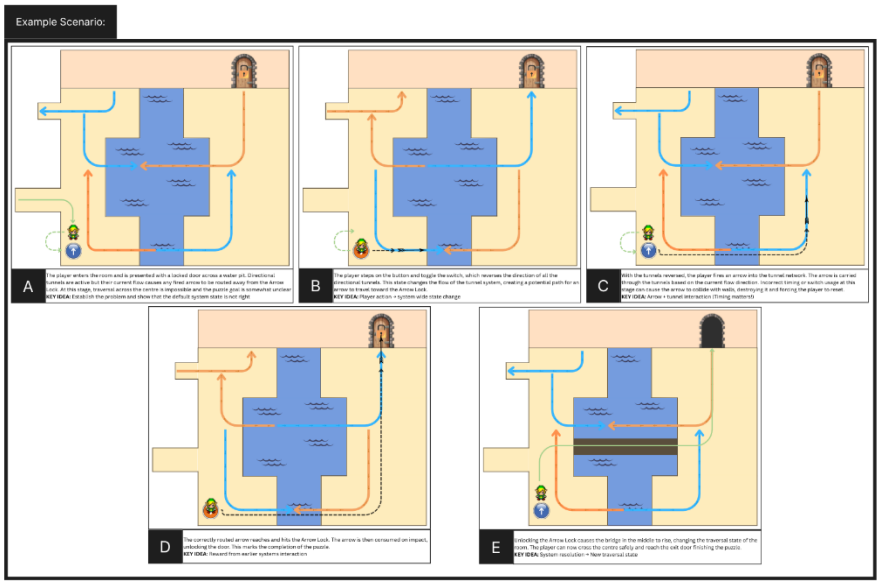
The emergent gameplay in this design comes from the relationships between all three objects:

- The Toggle Switch changes the state of the Directional Tunnels
- Directional Tunnels determine arrow routing outcomes
- Arrow routing determines whether the Arrow Lock can be activated

- Activating the Arrow Lock changes traversal possibilities within the room

None of these objects function as one complete solution to the puzzle on their own. Instead their meaning comes from their interactions across the player input. Incorrect timing, positioning, or understanding of the tunnel system results in failure, while correct sequencing rewards by success. This allows for the same systems to support multiple puzzle variants just by rearranging tunnel layouts, switch placement and locked door positions.

2.2. SCENERIO JUSTIFICATION:



The example scenario I made shows how the system evolves across different board states rather than resolving instantly. Each panel highlights a certain system state:

- **A** An initial incorrect configuration of the board
- **B** A player triggered state change and projectile throw
- **C** Projectile routing under the new conditions
- **D** Successful system interaction with projectile and Arrow Lock
- **E** A changed traversal state as the reward (Bridge opening up)

This progression shows how players understanding develops through interaction and feedback rather than explicit instructions.

2.3. ELEGANT DESIGN AND POSSIBLITY SPACE:

This design achieves a big gameplay possibility space using a very minimal number of objects by highlighting reusable properties and relationships. Directional Tunnels can be reused for different puzzle layouts, timing challenges, or traversal mechanics without introducing new mechanics.

Because the system relies on state changes rather than scripted sequences, designers can easily expand the design by adding more tunnels, switches, or environmental constraints while still preserving the players understanding.

3.0 SYSTEM IMPLEMENTATION REFLECTION

The original systemic design was made into a working implementation in Unity! The goal of the implementation was to keep the systemic relationships between the Directional Tunnels, Toggle Switch, and Arrow Lock while using Unity's component system and event architecture.

The Directional Tunnel system was added using trigger colliders that apply velocity to the projectile while it remains inside the tunnel. Each tunnel stores two possible directions (normal and reversed) and changes its behaviour based on the current system state. Visual feedback was added by color changes and arrow indicators so that the player can easily understand the flow direction.

The Toggle Switch was implemented using a UnityEvent<bool> When the player steps on the switch, it calls that event which tells the new state to all the connected Directional Tunnel objects. Each tunnel listens to the event and updates its direction, color, and arrow visuals. Allowing for a single switch to control multiple tunnels without needing direct references between scripts.

The Arrow Lock was added as a locked exit that uses an interface called IHittable. The projectile checks whether the object it collides with implements this interface and calls its Hit() method if it does. Allowing the projectile to interact with different objects without needing to know their specific type.

When the Arrow Lock is activated by the projectile, several event responses occur:

- The door changes from a locked sprite to an unlocked sprite
- The blocking collider is disabled so the player can pass through
- A bridge rises to open the path forward
- The directional tunnels are disabled since the puzzle was complete

These reactions to such a simple gameplay event show how something can spread through many different systems in the environment.

One minor system was added to improve gameplay, that being a reset trigger if the player falls into the water beneath the level, which resets the scene if done. Preventing soft lock situations and keeps the gameplay loop consistent.